



# K-Means Clustering — Practical Implementation



## Objective

The goal of this notebook is to understand how K-Means Clustering works through practical implementation and visualization.

Instead of focusing only on theory, this notebook aims to build intuition around:

- how clusters are formed
  - how centroids work
  - how points are assigned to clusters
  - and why distance plays a critical role in clustering
- 



## What is K-Means Clustering?

K-Means is an unsupervised machine learning algorithm used to group similar data points together.

The algorithm forms clusters based on similarity and distance between points.

---



## What Does K Represent?

In K-Means:

- **K** represents the number of clusters the algorithm should create.

For example:

- $K = 2 \rightarrow$  create 2 clusters
  - $K = 3 \rightarrow$  create 3 clusters
- 



## What Are Centroids?

Centroids are the central points of clusters.

The algorithm:

1. Creates centroids
  2. Assigns nearby points to the nearest centroid
  3. Recalculates centroid positions
  4. Repeats the process until the centroids stabilize
- 



## Real-Life Intuition

A simple real-world example is food delivery optimization.

Suppose a company wants to open 3 delivery hubs in a city.

The goal is to place hubs such that customers are closest to them.

K-Means helps identify these central hub locations and groups nearby customers together efficiently.

---



## Why K-Means Is Important

K-Means is widely used in:

- customer segmentation
- recommendation systems
- fraud detection
- image compression
- market analysis

because it helps discover hidden patterns and groups in data.



## Step 1 — Creating the Dataset

To understand how K-Means Clustering works, we first create a small customer-style dataset.

The dataset contains:

- Income
- Spending

These features help simulate customer purchasing behavior.

---

## Why This Dataset?

K-Means works by grouping similar data points together.

If customers have similar:

- income levels
- spending patterns

the algorithm should naturally place them into the same cluster.

---

## Real-World Intuition

This type of analysis is commonly used in shopping malls and e-commerce platforms to identify:

- premium customers
- budget customers
- high-spending groups
- low-engagement customers

without predefined labels.

---

## Expected Outcome

We expect the dataset to naturally form separate customer groups based on similarity and distance.

```
In [1]: # Create Dataset
```

```
In [2]: import pandas as pd

data = {
    'Income': [15, 16, 17, 18, 60, 62, 64, 65],
    'Spending': [20, 22, 19, 18, 80, 82, 78, 79]
}

df = pd.DataFrame(data)
```

df

Out[2]:

|   | Income | Spending |
|---|--------|----------|
| 0 | 15     | 20       |
| 1 | 16     | 22       |
| 2 | 17     | 19       |
| 3 | 18     | 18       |
| 4 | 60     | 80       |
| 5 | 62     | 82       |
| 6 | 64     | 78       |
| 7 | 65     | 79       |

🧠 What Are the 2 Natural Groups? Group 1 Customers with: \*lower income \*lower spending Group 2 Customers with: \*higher income \*higher spending

## Initial Observation Before Applying K-Means

Before applying the algorithm, the dataset already appears to contain two natural groups.

### Group 1

Customers with:

- lower income
- lower spending

### Group 2

Customers with:

- higher income
- higher spending

---

## Important Understanding

At this stage, there are no predefined labels or categories.

The algorithm does not know:

- which customers are premium
- which customers are budget customers

It only analyzes:

- similarity
- distance between data points

and attempts to discover hidden patterns automatically.

---



## Real-World Relevance

This is one of the major advantages of clustering algorithms.

Businesses can identify customer groups and behavioral patterns without manually labeling the data beforehand.

In [ ]:



## Step 2 — Visualizing the Dataset

Before applying K-Means Clustering, it is important to first visualize the dataset.

Visualization helps:

- understand data distribution
  - identify natural groupings
  - build intuition about similarity between points
- 



## Why Visualization Matters in Clustering

Clustering algorithms rely heavily on:

- distance
- similarity between points

By plotting the data, we can visually inspect whether similar points are naturally grouped together.

---

## Real-World Intuition

This is similar to observing customer distribution on a city map before deciding where to place delivery hubs or service centers.

Visualization helps identify:

- dense regions
- separated groups
- and possible cluster structures

before applying the algorithm.

---

## Expected Observation

We expect the dataset to visually form:

- one lower-income, lower-spending group
- one higher-income, higher-spending group

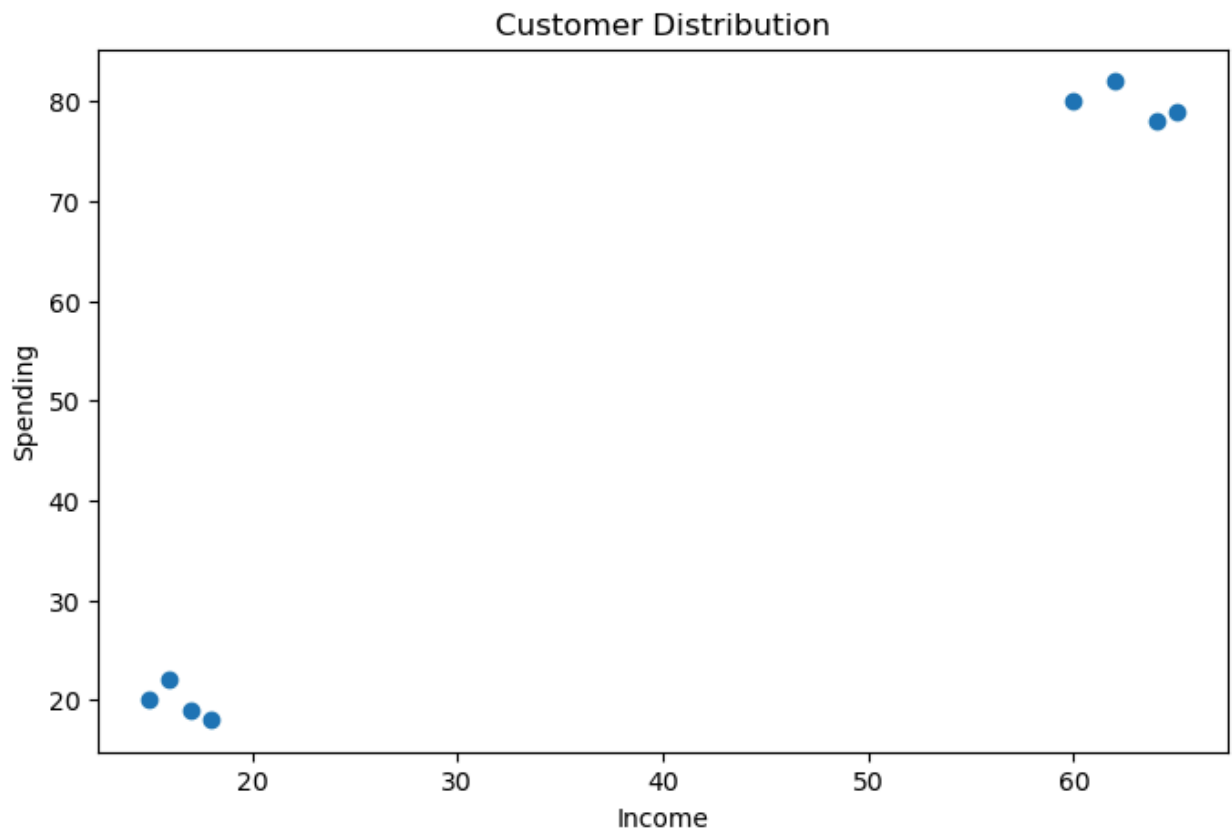
```
In [3]: import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))

plt.scatter(df['Income'], df['Spending'])

plt.xlabel("Income")
plt.ylabel("Spending")
plt.title("Customer Distribution")

plt.show()
```



## Observation After Visualizing the Dataset

After plotting the dataset, two clear groups become visible.

---



### Group 1

Customers with:

- lower income
- lower spending

These points appear closely packed together in the lower-left region of the graph.

---



### Group 2

Customers with:

- higher income
- higher spending

These points form another tightly grouped cluster in the upper-right region.

---

## Important Insight

The distance between the two groups is relatively large.

This indicates that the dataset already contains natural cluster structures, which should make it easier for K-Means to separate the points effectively.

---

## Real-World Interpretation

This type of grouping can help businesses identify:

- budget customers
- premium customers
- high-value customer segments

without manually defining categories beforehand.

---

## Key Learning

Even before applying K-Means, the hidden structure within the data becomes visible through visualization.

K-Means helps mathematically identify and formalize these natural groupings.

In [ ]:

## Step 3 — Applying K-Means Clustering

After visualizing the dataset, we now apply the K-Means algorithm to identify clusters mathematically.



The algorithm will:

- create centroids
  - assign nearby points to the nearest centroid
  - repeatedly adjust centroid positions
  - and continue until the clusters stabilize
- 



## Why K-Means Works

K-Means relies on:

- similarity
- distance between points

Points that are closer to the same centroid are grouped into the same cluster.

---



## Real-World Intuition

This is similar to placing delivery hubs in a city.

The algorithm attempts to place central locations such that nearby customers are assigned to the closest hub, improving efficiency and reducing distance.

---



## Choosing $K = 2$

Based on visualization, the dataset appeared to contain two natural groups:

- lower income & lower spending
- higher income & higher spending

Therefore:

- $K = 2$

is used to instruct the algorithm to form two clusters.

```
In [7]: import warnings
warnings.filterwarnings('ignore')
```

```
In [8]: from sklearn.cluster import KMeans

model = KMeans(n_clusters=2, random_state=42)

clusters = model.fit_predict(df)

df['Cluster'] = clusters

df
```

```
Out[8]:
```

|   | Income | Spending | Cluster |
|---|--------|----------|---------|
| 0 | 15     | 20       | 0       |
| 1 | 16     | 22       | 0       |
| 2 | 17     | 19       | 0       |
| 3 | 18     | 18       | 0       |
| 4 | 60     | 80       | 1       |
| 5 | 62     | 82       | 1       |
| 6 | 64     | 78       | 1       |
| 7 | 65     | 79       | 1       |



## Observation After Applying K-Means

After applying K-Means Clustering with:

- $K = 2$

the algorithm successfully separated the dataset into two clusters.

---



## Cluster Patterns Observed

### Cluster 1

Customers with:

- lower income
- lower spending

## Cluster 2

Customers with:

- higher income
  - higher spending
- 



## Important Insight

The algorithm was able to identify the same natural groups that were previously observed visually.

This indicates that the dataset contains strong cluster separation.

---



## Real-World Interpretation

This type of clustering can help businesses automatically identify:

- budget customer groups
- premium customer segments
- spending-based behavioral patterns

without manually labeling customers beforehand.

---



## Key Learning

K-Means does not understand business meaning or customer categories.

It only relies on:

- similarity
- and distance between points

to group data points together.



## Step 4 — Visualizing Clusters and Centroids

After applying K-Means Clustering, we now visualize:

- cluster assignments
- centroid locations
- and group separation

Visualization helps verify whether the algorithm successfully identified meaningful clusters.

---



### Understanding Centroids

A centroid represents the central point of a cluster.

It is calculated as the average position of all points belonging to that cluster.

The centroid is not necessarily an actual data point. Instead, it acts as the mathematical center of the cluster.

---



### Real-World Intuition

This can be compared to delivery hubs in a city.

Customers are assigned to the nearest hub, and the hub location represents the centroid of that customer group.

---



### Why Visualization Matters

By visualizing:

- clusters
- distances
- and centroid positions

it becomes easier to understand how K-Means forms groups internally.

---

## Expected Observation

We expect:

- two clearly separated customer groups
- and centroids positioned near the center of each cluster.

```
In [9]: import matplotlib.pyplot as plt

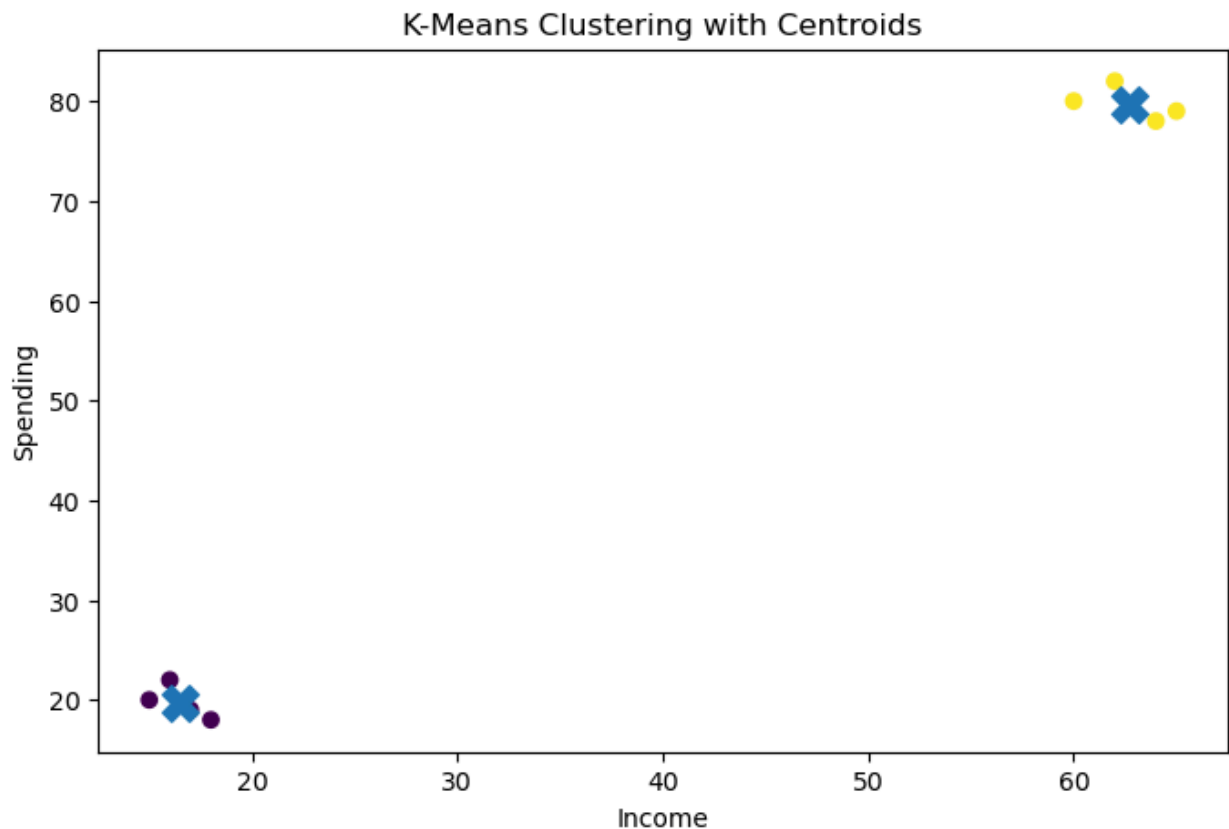
plt.figure(figsize=(8,5))

# Plot data points
plt.scatter(
    df['Income'],
    df['Spending'],
    c=df['Cluster']
)

# Plot centroids
plt.scatter(
    model.cluster_centers[:,0],
    model.cluster_centers[:,1],
    s=200,
    marker='X'
)

plt.xlabel("Income")
plt.ylabel("Spending")
plt.title("K-Means Clustering with Centroids")

plt.show()
```



## Observation After Visualizing Centroids

After plotting the clusters and centroids, the centroid positions appeared near the center of each group.

This happened because K-Means continuously adjusts centroid locations to minimize the overall distance between:

- data points
- and their assigned centroid.

---

## Understanding Centroid Behavior

Initially, centroids are placed randomly.

The algorithm then repeatedly:

1. assigns points to the nearest centroid

2. recalculates the cluster center
  3. moves the centroid
  4. repeats the process until the centroids stabilize
- 

## Important Insight

The centroid represents the most central and representative location of a cluster.

Its goal is to minimize the distance between itself and all points belonging to that cluster.

---

## Real-World Interpretation

This is similar to determining the best location for:

- delivery hubs
- warehouses
- service centers

where the goal is to minimize the travel distance for nearby customers.

---

## Key Learning

K-Means works by minimizing within-cluster distance, ensuring that similar points remain close to their cluster centroid.

## Moving from Simple Clusters to Real-World Clustering

Until this point, the dataset contained very clearly separated groups:

- low income & low spending
- high income & high spending

Because the clusters were highly distinct, both visualization and clustering became

relatively straightforward.

---



## But Real-World Data Is Rarely This Clean

In real-world scenarios:

- customer behavior overlaps
- patterns are noisy
- boundaries between groups are unclear
- and clusters are often difficult to identify visually

For example:

- a customer may have high income but low spending
- another customer may have medium income but luxury spending behavior

This creates mixed and overlapping patterns within the dataset.

---



## Why This Is Important

The earlier dataset helped build intuition around:

- centroids
- distance
- cluster formation
- and K-Means workflow

However, real clustering problems become challenging because the hidden structure is not immediately visible.

---



## Real-World Relevance

In actual business scenarios, customer behavior is rarely perfectly separated.

Companies often deal with:



- overlapping customer groups
- inconsistent purchasing behavior
- noisy transactional patterns
- and dynamic user preferences

This is where clustering becomes truly valuable because the algorithm attempts to discover hidden structures within messy data.

---

## Next Objective

The next step is to work with a more realistic dataset containing:

- overlapping behaviors
- mixed spending patterns
- and less obvious cluster boundaries

This will help us understand how K-Means behaves in practical real-world situations.

```
In [10]: import pandas as pd

data = {
    'Income': [15, 18, 20, 22, 25, 30, 35, 40,
               45, 50, 52, 55, 58, 60, 62, 65,
               68, 70, 72, 75],
    'Spending': [20, 25, 18, 30, 35, 40, 38, 45,
                 50, 48, 60, 55, 52, 65, 70, 68,
                 72, 60, 75, 80]
}

df_real = pd.DataFrame(data)

df_real
```

Out[10]:

|           | Income | Spending |
|-----------|--------|----------|
| <b>0</b>  | 15     | 20       |
| <b>1</b>  | 18     | 25       |
| <b>2</b>  | 20     | 18       |
| <b>3</b>  | 22     | 30       |
| <b>4</b>  | 25     | 35       |
| <b>5</b>  | 30     | 40       |
| <b>6</b>  | 35     | 38       |
| <b>7</b>  | 40     | 45       |
| <b>8</b>  | 45     | 50       |
| <b>9</b>  | 50     | 48       |
| <b>10</b> | 52     | 60       |
| <b>11</b> | 55     | 55       |
| <b>12</b> | 58     | 52       |
| <b>13</b> | 60     | 65       |
| <b>14</b> | 62     | 70       |
| <b>15</b> | 65     | 68       |
| <b>16</b> | 68     | 72       |
| <b>17</b> | 70     | 60       |
| <b>18</b> | 72     | 75       |
| <b>19</b> | 75     | 80       |

## Initial Observation on the Realistic Dataset

Unlike the earlier dataset, the customer patterns in this dataset are much less obvious.

The data appears:

- more cluttered
  - more overlapping
  - and less clearly separated
-



## Important Observation

There no longer appears to be a perfectly defined number of clusters.

Instead of two obvious groups, the dataset now contains:

- mixed customer behavior
- medium-range spending patterns
- and overlapping regions

This makes the clustering problem much more realistic.

---



## Real-World Interpretation

In practical business scenarios:

- customer behavior rarely follows perfectly separated categories
- many customers fall between different behavioral groups
- and patterns often overlap significantly

This creates uncertainty while deciding:

- how many clusters actually exist
  - and where the boundaries should be drawn
- 



## Key Learning

This dataset better represents real-world clustering problems where hidden patterns are difficult to identify visually.

It also highlights one of the biggest challenges in K-Means:

- choosing the correct value of K.



## Visualizing the Realistic Dataset

After creating the realistic customer dataset, the next step is to visualize the

distribution of points.

Visualization helps identify:

- overlapping behavior
  - dense regions
  - possible cluster structures
  - and unclear boundaries between groups
- 



## Why Visualization Is Important

In real-world clustering problems:

- clusters are rarely perfectly separated
- customer behavior often overlaps
- and hidden patterns are difficult to identify directly

Visual inspection helps build intuition before applying clustering algorithms.

---



## Real-World Interpretation

This is similar to observing customer behavior inside a shopping mall.

Some customer groups may appear:

- highly similar
- tightly packed
- and clearly separated

while others may:

- overlap significantly
  - behave inconsistently
  - or fall between multiple behavioral groups
- 



## Expected Observation

Unlike the earlier clean dataset, this visualization is expected to show:

- overlapping regions
- mixed behavioral patterns
- and less obvious cluster separation

This better represents practical real-world clustering scenarios.

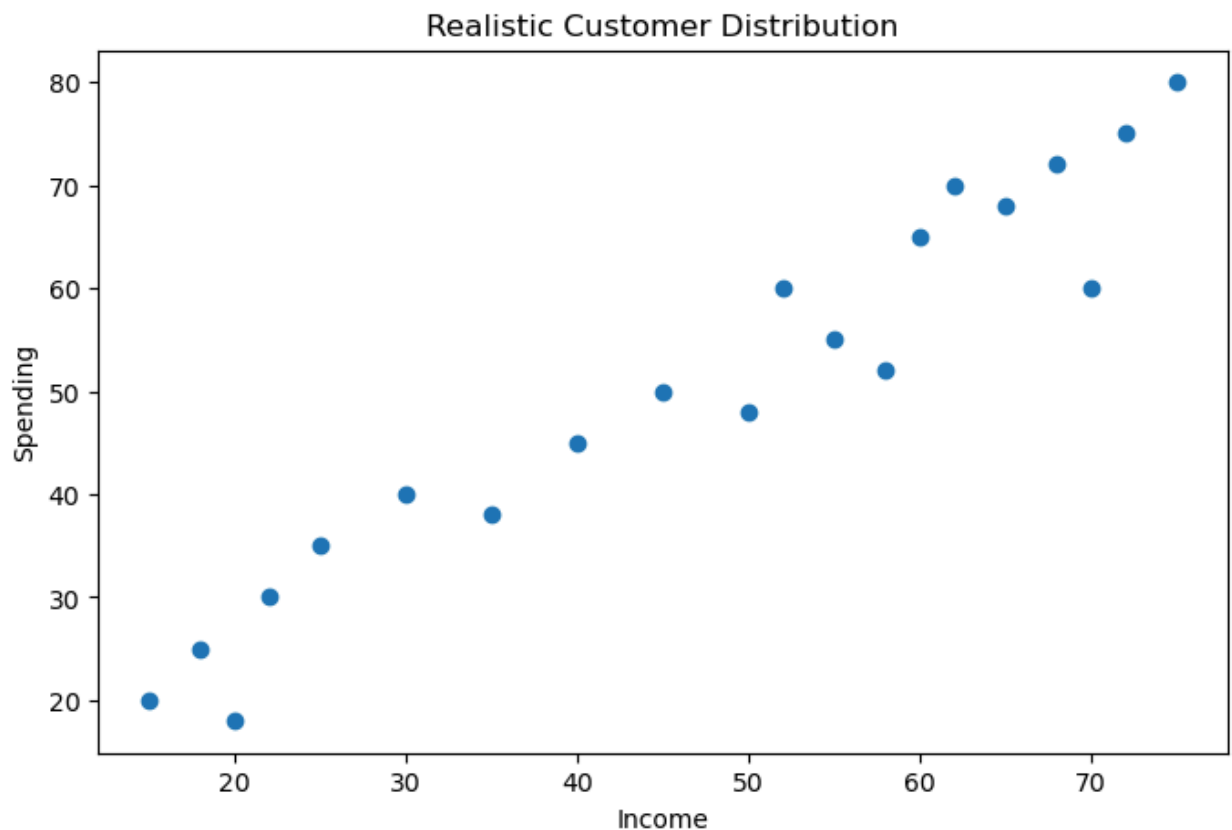
```
In [11]: import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))

plt.scatter(
    df_real['Income'],
    df_real['Spending']
)

plt.xlabel("Income")
plt.ylabel("Spending")
plt.title("Realistic Customer Distribution")

plt.show()
```



## Observation After Visualizing the Realistic Dataset

The realistic dataset appears significantly more diverse compared to the earlier example.

Instead of showing two clearly separated groups, the graph now contains:

- overlapping behavior
  - gradual transitions between points
  - and less obvious cluster boundaries
- 

## Important Insight

The dataset now appears more continuous rather than perfectly separated.

This indicates that customer behavior exists across a spectrum instead of forming isolated categories.

Some customers may:

- spend heavily despite moderate income
  - earn highly but spend conservatively
  - or fall between multiple behavioral patterns
- 

## Real-World Interpretation

This type of overlap is extremely common in real business datasets.

Customer behavior is rarely divided into perfectly distinct groups.

Instead, businesses often deal with:

- mixed behavioral patterns
  - overlapping segments
  - and ambiguous customer profiles
-



## Key Learning

This dataset better represents practical clustering problems where:

- the number of clusters is not obvious
- cluster boundaries are unclear
- and hidden patterns must be discovered mathematically.



## Understanding the Elbow Method

One of the biggest challenges in K-Means Clustering is choosing the correct value of:

- K (number of clusters)

Unlike the earlier dataset where the groups were visually obvious, real-world datasets often contain overlapping patterns and unclear cluster boundaries.

---



## Why Choosing K Is Difficult

If K is too small:

- very different points may be forced into the same cluster

If K is too large:

- clusters may become overly fragmented and less meaningful

Therefore, selecting the correct number of clusters becomes extremely important.

---



## What Is WCSS?

K-Means calculates a metric called:

- WCSS (Within-Cluster Sum of Squares)

WCSS measures how far data points are from their assigned centroid.

Smaller WCSS values indicate:

- tighter clusters
  - and better compactness within groups.
- 



## Important Observation

As K increases:

- WCSS always decreases

because more clusters reduce the average distance between points and centroids.

However, continuously increasing K is not practical because clusters may become unnecessarily fragmented.

---



## What Is the Elbow Method?

The Elbow Method helps identify the optimal value of K by plotting:

- number of clusters (K) vs
- WCSS values

The ideal K is usually found at the “elbow point” where:

- the rate of WCSS improvement starts slowing down significantly.
- 



## Real-World Intuition

This is similar to opening delivery hubs in a city.

Adding a second or third hub may greatly improve delivery efficiency, but after a certain point, adding more hubs provides only minor improvements while increasing complexity and cost.

```
In [12]: from sklearn.cluster import KMeans

wcss = []

for k in range(1, 11):
```



```
model = KMeans(  
    n_clusters=k,  
    random_state=42  
)  
  
model.fit(df_real)  
  
wcss.append(model.inertia_)
```

## Observation After Calculating WCSS

K-Means was executed multiple times using different values of:

- $K = 1$  to 10

For each iteration, the algorithm calculated:

- WCSS (Within-Cluster Sum of Squares)

which measures how closely data points are grouped around their assigned centroid.

---

## Important Insight

As the number of clusters increases:

- WCSS decreases

This happens because:

- more centroids reduce the average distance between points and their assigned cluster center.
- 

## However

Continuously increasing the number of clusters is not always useful.

Although WCSS improves, the improvement eventually becomes smaller and less meaningful.

This creates the need to identify:

- the point where improvement begins slowing significantly

which is known as:

- the elbow point.
- 



## Real-World Interpretation

This is similar to opening additional delivery hubs in a city.

Initially:

- adding more hubs significantly improves efficiency

But after a certain point:

- additional hubs provide only minor improvements while increasing complexity and operational cost.



## Visualizing the Elbow Curve

After calculating WCSS values for different cluster counts, the next step is to visualize the relationship between:

- number of clusters (K)
- and WCSS

This visualization is called:

- the Elbow Curve
- 



## Why the Elbow Curve Matters

As K increases:

- WCSS decreases

because clusters become tighter and data points move closer to their assigned

centroid.

However, after a certain point:

- the improvement becomes much smaller

This slowdown in improvement creates a bend in the graph known as:

- the elbow point
- 

## Goal of the Elbow Method

The objective is to identify the point where:

- adding more clusters no longer provides significant improvement

This point often represents the most balanced and meaningful choice for K.

---

## Real-World Interpretation

This is similar to opening additional delivery hubs in a city.

Initially:

- adding hubs greatly improves efficiency

But after a certain point:

- additional hubs provide only minor improvements while increasing operational complexity and cost.

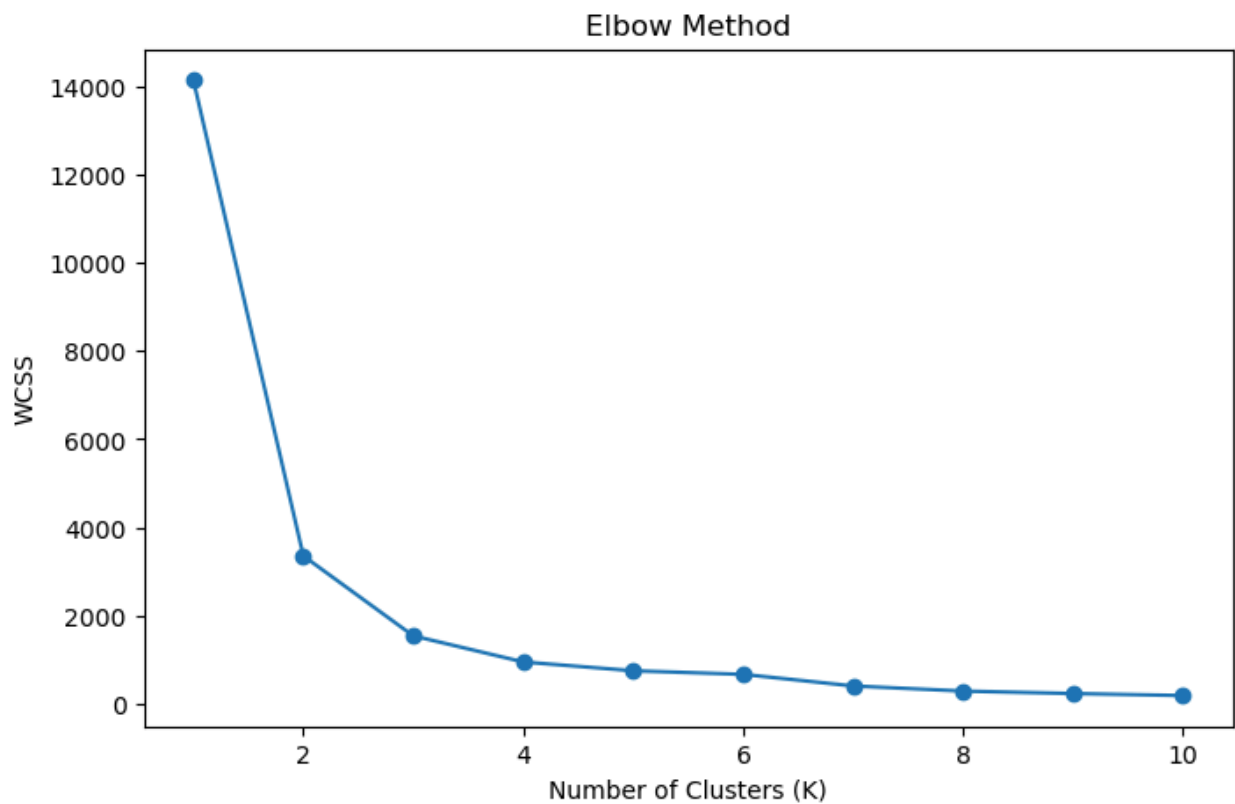
```
In [13]: import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))

plt.plot(range(1, 11), wcss, marker='o')

plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.title("Elbow Method")

plt.show()
```



## Observation from the Elbow Curve

The elbow curve shows a sharp decrease in WCSS during the initial cluster increases.

---



## Important Observation

From:

- $K = 1$  to  $K = 2$

there is a very large improvement in cluster compactness.

However:

- the curve still continues to improve significantly from  $K = 2$  to  $K = 3$ .

After:

- $K = 3$

the curve begins to flatten and the improvement in WCSS becomes much smaller.

---



## Estimated Elbow Point

Based on the visualization, the elbow point appears to be near:

- $K = 3$

This suggests that dividing the dataset into 3 clusters may provide a balanced tradeoff between:

- cluster compactness
  - and model simplicity.
- 



## Important Learning

The elbow point is not always perfectly sharp or obvious.

In real-world datasets:

- multiple  $K$  values may appear reasonable
  - and selecting the optimal number of clusters often requires:
    - domain understanding
    - visualization
    - and additional evaluation techniques.
- 



## Real-World Interpretation

This is similar to adding delivery hubs in a city.

Initially:

- adding more hubs greatly improves efficiency

But after a certain point:

- additional hubs provide only small improvements while increasing complexity and operational cost.

# Applying K-Means with the Optimal K Value

Based on the Elbow Method, the elbow point appeared near:

- $K = 3$

This suggests that dividing the dataset into three clusters may provide a balanced tradeoff between:

- compact clusters
  - and model simplicity.
- 

## Why This Step Is Important

Unlike the earlier clean dataset, the realistic dataset contains:

- overlapping behavior
- gradual transitions
- and less obvious cluster boundaries

This makes clustering more practical and challenging.

---

## Real-World Interpretation

In a real shopping mall scenario, customers may naturally form groups such as:

- budget customers
- moderate spenders
- premium customers

K-Means attempts to identify these hidden behavioral segments automatically using similarity and distance calculations.

---

## Expected Observation

Since the dataset is more realistic:

- some overlap between clusters may occur
- certain points may appear ambiguous
- and centroid balancing becomes more noticeable

This better represents how clustering behaves in practical real-world situations.

```
In [14]: from sklearn.cluster import KMeans

model_real = KMeans(
    n_clusters=3,
    random_state=42
)

clusters_real = model_real.fit_predict(df_real)

df_real['Cluster'] = clusters_real

df_real
```

Out[14]:

|           | Income | Spending | Cluster |
|-----------|--------|----------|---------|
| <b>0</b>  | 15     | 20       | 2       |
| <b>1</b>  | 18     | 25       | 2       |
| <b>2</b>  | 20     | 18       | 2       |
| <b>3</b>  | 22     | 30       | 2       |
| <b>4</b>  | 25     | 35       | 2       |
| <b>5</b>  | 30     | 40       | 2       |
| <b>6</b>  | 35     | 38       | 2       |
| <b>7</b>  | 40     | 45       | 0       |
| <b>8</b>  | 45     | 50       | 0       |
| <b>9</b>  | 50     | 48       | 0       |
| <b>10</b> | 52     | 60       | 0       |
| <b>11</b> | 55     | 55       | 0       |
| <b>12</b> | 58     | 52       | 0       |
| <b>13</b> | 60     | 65       | 1       |
| <b>14</b> | 62     | 70       | 1       |
| <b>15</b> | 65     | 68       | 1       |
| <b>16</b> | 68     | 72       | 1       |
| <b>17</b> | 70     | 60       | 1       |
| <b>18</b> | 72     | 75       | 1       |
| <b>19</b> | 75     | 80       | 1       |

## Observation After Applying K-Means with $K = 3$

After applying K-Means with:

- $K = 3$

the dataset was divided into three separate customer groups.

---





## Important Insight

Unlike the earlier dataset that contained only two highly distinct groups, the realistic dataset produced more layered segmentation.

The algorithm was able to identify:

- lower-range customer behavior
  - intermediate customer patterns
  - and higher-value customer groups
- 



## Real-World Interpretation

This type of segmentation is much more realistic for practical business applications.

Instead of simply separating customers into:

- low spending
- and high spending

the algorithm now captures:

- moderate customers
  - transitional behaviors
  - and mixed spending patterns as separate groups.
- 



## Important Learning

Real-world clustering is rarely perfectly separated.

Some data points may:

- overlap between clusters
- appear ambiguous
- or lie near cluster boundaries

This is normal because clustering is based on:

- similarity
- and distance calculations

rather than predefined labels.

---



## Key Learning

K-Means attempts to minimize within-cluster distance and create meaningful groupings even when the data contains overlapping and complex patterns.



## Visualizing Realistic Clusters and Centroids

After applying K-Means with:

- $K = 3$

the next step is to visualize:

- cluster assignments
  - customer distribution
  - and centroid positions
- 



## Why This Visualization Is Important

Unlike the earlier clean dataset, the realistic dataset contains:

- overlapping behavior
- mixed customer patterns
- and less obvious boundaries

Visualization helps us understand how K-Means behaves when the data is more complex and practical.

---

## Real-World Interpretation

In real customer segmentation problems:

- not every customer clearly belongs to a single category
- some customers may fall between groups
- and behavioral overlap is very common

K-Means still attempts to create the most balanced grouping possible using distance and centroid optimization.

---

## Expected Observation

The visualization is expected to show:

- less perfect cluster separation
  - centroid balancing across regions
  - and more natural-looking customer segmentation patterns.
- 

## Key Learning

Real-world clustering is rarely perfectly clean.

The algorithm attempts to organize messy and overlapping data into meaningful groups while minimizing within-cluster distance.

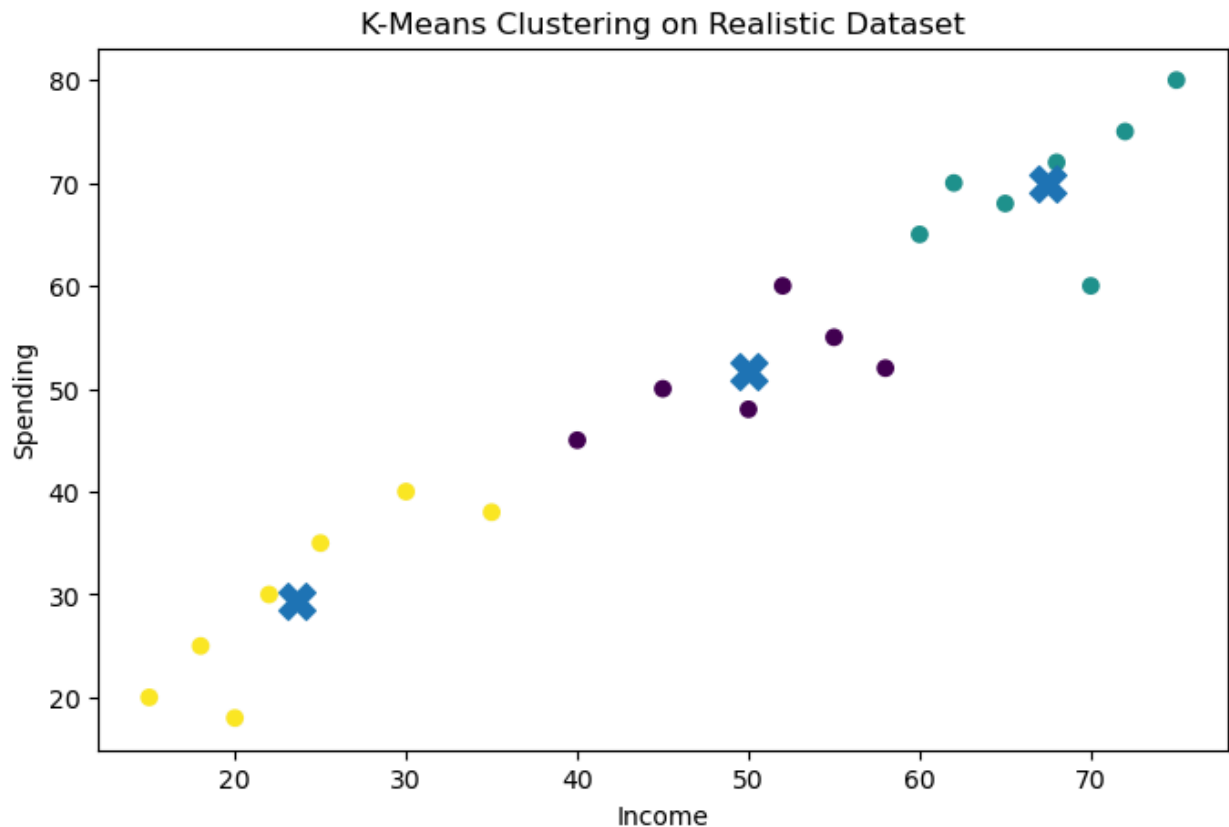
```
In [15]: import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))

# Plot clustered points
plt.scatter(
    df_real['Income'],
    df_real['Spending'],
    c=df_real['Cluster']
)

# Plot centroids
plt.scatter(
    model_real.cluster_centers_[ :,0],
    model_real.cluster_centers_[ :,1],
```

```
s=200,  
marker='X'  
)  
  
plt.xlabel("Income")  
plt.ylabel("Spending")  
plt.title("K-Means Clustering on Realistic Dataset")  
  
plt.show()
```



## Observation After Visualizing Realistic Clusters

After applying K-Means with:

- $K = 3$

the dataset was divided into three separate customer groups with centroids positioned near the center of each cluster.

---



## Important Insight

Unlike the earlier clean dataset, the clusters now appear more natural and realistic.

Instead of perfectly isolated groups, the dataset contains:

- gradual behavioral transitions
- overlapping regions
- and mixed customer patterns

The centroids attempt to balance these regions by positioning themselves near the most representative center of each cluster.

---



## Understanding Centroid Behavior

The centroid does not represent an actual customer.

Instead, it acts as the mathematical center of a cluster and attempts to minimize the overall distance between itself and nearby points.

---



## Real-World Interpretation

This type of clustering is much closer to practical business segmentation where customers may naturally fall into groups such as:

- low-value customers
- moderate customers
- premium customers

without having perfectly clear boundaries between them.

---



## Key Learning

Real-world clustering focuses on finding the best possible grouping based on similarity and distance rather than creating perfectly separated categories.

# ⚠ Limitations of K-Means Clustering

Although K-Means is one of the most widely used clustering algorithms, it also has several practical limitations.

Working with the realistic dataset helped reveal some of these challenges more clearly.

---

## 1. Choosing the Correct Value of K Is Difficult

One of the biggest limitations of K-Means is that the number of clusters must be specified beforehand.

In real-world datasets:

- the correct number of groups is often unknown
- cluster boundaries may overlap
- and multiple K values may appear reasonable

This makes cluster selection subjective and challenging.

---

## 2. Sensitive to Initial Centroid Placement

K-Means begins with randomly initialized centroids.

Different centroid starting points can sometimes produce:

- different cluster structures
- different segmentation patterns
- and inconsistent results

This is why parameters like:

- `random_state`

are important for reproducibility.

---



## 3. Struggles with Overlapping Data

In realistic datasets:

- customer behavior often overlaps
- boundaries are unclear
- and some points may belong near multiple clusters

K-Means still forces every point into a cluster, even when separation is not perfectly clear.

---



## 4. Sensitive to Outliers

Extreme values or unusual points can heavily influence centroid positions.

Since centroids are based on averages, outliers may distort cluster formation and reduce clustering quality.

---



## 5. Assumes Roughly Spherical Clusters

K-Means works best when clusters are:

- compact
- evenly distributed
- and roughly circular in shape

It may struggle with:

- irregular cluster structures
  - elongated patterns
  - or highly complex distributions
-

## Real-World Understanding

Despite these limitations, K-Means remains extremely useful because:

- it is simple
  - computationally efficient
  - easy to interpret
  - and highly effective for many practical business segmentation problems.
- 

## Final Learning

K-Means is not about finding perfect categories.

It attempts to create the most balanced grouping possible based on:

- similarity
- distance
- and centroid optimization.

## Practical Business Application of K-Means Clustering

Until this point, clustering was performed mathematically using:

- distance
- centroids
- and similarity between points.

However, the real value of K-Means appears when clusters are interpreted from a business perspective.

---

## Business Problem

Suppose a shopping mall or e-commerce company wants to better understand customer behavior.



The company may ask questions such as:

- Which customers spend the most?
  - Which customers are budget-conscious?
  - Which customers may respond to premium offers?
  - Which customer groups require different marketing strategies?
- 

## How K-Means Helps

K-Means automatically groups customers with similar behavior patterns together.

Instead of manually categorizing users, the algorithm identifies hidden customer segments using:

- income
  - spending behavior
  - and similarity between customers
- 

## Real-World Applications

Customer segmentation is widely used in:

- e-commerce platforms
- banking
- retail stores
- subscription services
- streaming platforms
- and digital marketing systems

to:

- personalize recommendations
  - improve targeting
  - optimize promotions
  - and increase customer retention.
- 

```
In [16]: # Lets analyze clusters Practically
```

```
df_real.groupby('Cluster').mean()
```

Out[16]:

|         | Income    | Spending  |
|---------|-----------|-----------|
| Cluster |           |           |
| 0       | 50.000000 | 51.666667 |
| 1       | 67.428571 | 70.000000 |
| 2       | 23.571429 | 29.428571 |

## Interesting Behavioral Observation

One interesting observation from the clustering results is that:

- Cluster 2 contains customers with relatively lower income
  - but comparatively higher spending behavior.
- 

## Important Understanding

This does not necessarily mean that customers are literally spending more money than they earn.

Instead, it indicates that:

- their spending activity is relatively high compared to their income level
  - making their purchasing behavior more aggressive or engagement-heavy.
- 

## Possible Real-World Interpretations

This type of customer behavior may represent:

- impulsive buyers
- emotionally driven customers
- highly engaged shoppers
- or customers strongly influenced by promotions and discounts.

In practical business scenarios, companies may further investigate:

- purchase frequency

- payment behavior
- product preferences
- or retention patterns

to better understand this segment.

---

## Important Learning

Clustering algorithms do not provide final business answers.

Instead, they help identify:

- unusual patterns
- hidden customer behavior
- and segments that require deeper business analysis.

This demonstrates how machine learning supports decision-making by uncovering insights that may not be immediately visible.

By: Shubham Kulkarni

In [ ]: